

Andes: GPU-Native Tasking for Data-Dependent Parallelism

Anonymous Authors

Abstract—GPUs offer massive parallelism, yet writing high-performance kernels remains a significant challenge. This is especially true for problems with *irregular parallelism* that do not map cleanly to the GPU execution model.

We argue a tasking system is needed: a programming model that lets developers express problems as tasks and dependencies and a runtime that maps these tasks onto GPU hardware efficiently. To this end, we present Andes, a tasking system for GPUs comprising two components. The first is a lightweight bulk-synchronous runtime that supports arbitrary and recursive parallelism while automatically managing allocation, coherence, and scheduling. The second is a dataflow programming model called *flows* in which users specify serial chains of tasks that Andes optimizes for GPU Execution. On twelve real-world benchmarks, Andes achieves a geometric mean speedup of $2.18\times$ over the state-of-the-art GPU tasking framework CUDASTF and $2.65\times$ over state-of-the-art custom solutions.

I. INTRODUCTION

GPUs offer an order of magnitude more parallelism than CPUs, yet writing complex algorithmic tasks that fully exploit this parallelism remains one of the central challenges in high-performance computing. Achieving high GPU performance demands fine-grained control over the hardware: programmers must manually partition work across thread hierarchies (warps/blocks), allocate and stage data in shared memory, and tune tile sizes and block dimensions to match the specific memory access patterns of each algorithm.

This problem is compounded for irregular and data dependent algorithms where the available parallelism and control flow evolve dynamically at runtime. We refer to this class of workloads as **data-dependent parallel**: the structure and amount of parallel work available depend on intermediate program state and cannot be determined a priori. Examples include graph algorithms (e.g., SSSP, BFS), recursive numerical methods (e.g., Karatsuba multiplication), and combinatorial search (e.g., N -Queens). These workloads have sufficient parallelism but their irregular, dynamically evolving structure, i.e., the number and shape of tasks changes as execution proceeds, fundamentally conflict with the bulk-synchronous, single-program multiple-data execution model of GPUs.

On CPUs, data-dependent parallelism is addressed efficiently via **tasking**: work is subdivided into discrete, independent units called *tasks* that form a directed acyclic dependency graph (DAG), and a runtime system schedules them for execution as soon as their dependencies are satisfied [5], [6], [11]. However, porting tasking to GPUs requires overcoming a fundamental trade-off: GPUs offer massive parallelism but were designed for regular, statically-structured computation,

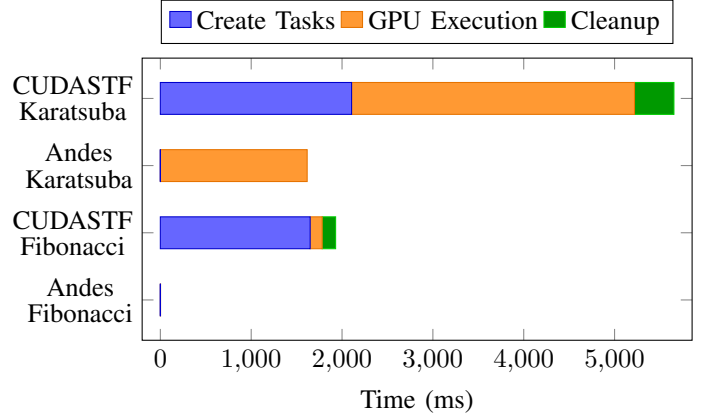


Fig. 1: Timing breakdown comparison between CUDASTF and Andes when solving a Karatsuba multiplication problem and calculating the 26^{th} Fibonacci number.

	CUDASTF	Andes
Karatsuba	28,769	116
26^{th} Fibonacci	196,418	56

TABLE I: Number of kernel launches when recursively calculating Karatsuba multiplication of two 2^{25} bit numbers and 26^{th} Fibonacci number.

not the dynamic scheduling and irregular control flow that tasking demands.

Existing tasking systems interact with GPUs in two ways: CPU-resident systems (e.g., TaskFlow [11], StarPU [2], UPC++ [3]) treat the GPU as a coarse-grained accelerator, limiting tasks to kernel-sized units and incurring frequent host-device synchronization. GPU-resident systems (e.g., Juggler [4], CUDASTF [1]) avoid this overhead by executing task graphs on the device using CUDA graphs, but require the task graph to be *statically constructed*. Specifically, CUDASTF [1], the current state of the art, implements the Sequential Task Flow (STF) model over CUDA APIs, automatically inferring task graphs from data dependencies and supporting both stream and CUDA graph backends. CUDASTF efficiently processes task graphs with low latency, making them an ideal choice for problems with a static launch structure such as SpGEMM [9], miniWeather [15], and prime sieving [10].

However, this reliance on static task graphs fundamentally limits existing GPU runtime systems as the task graph cannot be known in advance for data-dependent workloads and must instead be constructed dynamically during execution.

This limitation leads to significant overheads in practice,

as the host CPU must trace out the full parallel structure a priori to construct the task graph. Figure 1 and Table I show that for data-dependent workloads such as Karatsuba multiplication and computing Fibonacci numbers, CUDASTF spends a considerable fraction of execution time constructing the task graph. In Karatsuba, graph construction time on host approaches the cost of GPU computation, and the resulting CUDA graph contains over $\sim 28,000$ kernel launches. In fact, for fine-grained workloads such as Fibonacci, graph construction dominates execution time entirely.

This paper. We present **Andes**, a GPU-resident task runtime designed specifically for data-dependent parallelism. The key insight behind Andes is to decouple task representation from static graph structure: instead of pre-constructing the complete task graph, Andes dynamically creates, schedules, and executes tasks entirely on the GPU. Andes achieves this through the following core ideas:

- **Task virtualization**, which represents tasks as dynamically dispatchable functions, enabling heterogeneous task types within a single kernel.
- **Bulk-synchronous execution model**, which uses kernel boundaries as global synchronization points to ensure correctness under the GPU memory model.
- **Task fusion**, which dynamically merges compatible tasks to reduce kernel launch overhead.
- **Efficient scheduling**: a scalable GPU-resident executor paired with a GPU-native allocator dynamically manage task creation, execution, and cleanup without host intervention.

We evaluate Andes on a suite of twelve high-performance computing applications spanning graph analytics, numerical methods, and combinatorial search on modern NVIDIA GPUs. Across this suite, Andes achieves a geometric mean speedup of $2.65\times$ over hand-optimized GPU implementations and outperforms CUDASTF, the state-of-the-art GPU runtime, on data-dependent workloads with a geometric mean speedup of $2.18\times$ on Nvidia Blackwell RTX 6000 Pro. For workloads with static task structure, CUDA graph-based approaches remain competitive, highlighting the complementary strengths of dynamic and static tasking models.

In summary, this paper makes the following contributions:

- 1) A bulk-synchronous GPU task runtime that dynamically supports data-dependent task graphs without requiring a pre-declared static graph structure (Section II).
- 2) A task fusion mechanism that reduces kernel launch overhead by merging compatible tasks into a single launch at each phase boundary (Section II).
- 3) A task executor with an ensemble scheduling design that provides efficient, scalable scheduling without per-thread queues (Section III).
- 4) A custom API called *flows* that allows users to write GPU tasks without knowledge of the GPU runtime, parallelism, or memory coherency (Section IV).
- 5) A comprehensive evaluation across twelve irregular applications demonstrating competitive or superior performance compared to CUDASTF and state-of-the-art hand-

crafted solvers (Section V).

II. MODELING A TASK RUNTIME ON GPUS

The two critical constraints for a GPU task runtime to overcome are: 1) work must be launched as kernels and, 2) memory ordering across shared multiprocessors (SM) is weak by default. Andes overcomes these challenges via two design elements. First, we introduce a **bulk-synchronous execution model** that uses kernel closure as a synchronization barrier between task phases. Second, we utilize **dynamic dispatch** to run heterogeneous tasks within a single kernel launch. Based on these principles, we construct an executor that organizes ready tasks into a compact set of global queues bounded by a shared memory pool. We describe the two design elements in this section and discuss the executor in the next section.

A. Bulk synchronous execution

Efficient task execution on GPUs is fundamentally constrained by weak memory ordering, high synchronization costs, and the need for kernel-level resource specialization. These constraints make directly porting CPU-style work-stealing runtimes [11], [5], [3] to GPUs both inefficient and difficult to reason about.

To overcome these constraints, Andes adopts a **bulk-synchronous** execution model. Tasks are organized into explicit phases, and execution proceeds in globally synchronized steps: tasks in phase $i + 1$ do not begin until all tasks in phase i have completed. This is shown in Figure 2 where tasks are grouped into distinct rounds based on their dependencies. This design departs from the traditional work-stealing runtimes which are widely used on CPUs. The bulk-synchronous design aligns well with GPU execution model. Kernel boundaries provide global synchronization between rounds of tasks, abstracting away GPU memory semantics, enabling deterministic execution, and preserving opportunities for kernel-level optimizations.

We argue that work stealing is poorly suited to GPUs due to the synchronization and memory ordering overheads. Work stealing requires explicit synchronization between all execution units, meaning all tasks and all loads must be synchronized. This significantly complicates the execution of tasks and exacerbates the cost of loading memory and performing memory transactions inside of tasks.

B. Dynamic dispatch

GPU task runtimes face a fundamental trade-off between static specialization and dynamic dispatch. Current state-of-the-art systems such as CUDASTF [1] adopt static specialization, generating one kernel per task type and embedding task logic directly into the kernel. This design maximizes performance by eliminating indirect function calls, enabling inlining and aggressive compiler optimization.

However, static specialization requires that the structure of the task graph be known at kernel launch time. This limits the ability to support data-dependent workloads where new tasks can be generated recursively at runtime.

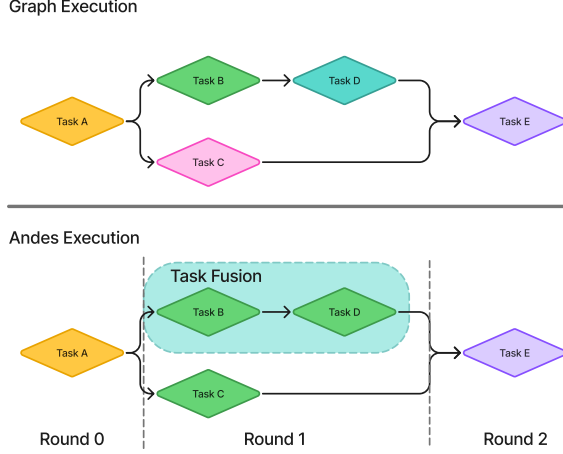


Fig. 2: CUDA graph-based execution in CUDASTF vs BSP execution in Andes. In the graph execution, every task is encapsulated as an independent CUDA kernel, with dependency resolution being handled by the scheduler. In this example, five kernel launches are executed, one for each task. In the Andes, tasks are executed in waves, with dynamic dispatch allowing for multiple task types to be run in one kernel. Task fusion compresses serial dependencies. In this example, all tasks can be processed in three kernel launches.

Andes adopts dynamic dispatch to efficiently support applications with irregular recursive workloads. In this model, tasks expose a shared virtual interface, allowing multiple task types to execute within a single kernel and new tasks to be created without returning to the host. The appropriate task implementation is resolved at runtime through virtual function calls. This allows more expressive task specifications, though the model does introduce slight additional overheads for indirect function calls and limits the ability of the compiler to inline. As we show in Section V, this flexibility is critical for achieving high throughput on irregular workloads and outweighs the per-task overheads of dynamic dispatch.

We next describe how this execution model is realized in the Andes executor.

III. EXECUTOR DESIGN

As shown in Section II, a bulk synchronous-based runtime promises high performance for GPU tasking while still allowing for arbitrary task execution. The Andes **executor** implements this bulk-synchronous, phase-driven execution model and is tailored to the GPU architecture. Execution proceeds in a sequence of *phases*, where each phase processes all tasks whose dependencies are satisfied.

Figure 2 showcases the Andes phased execution. In graph-based execution model in CUDASTF, each task is its own kernel which can be scheduled independently; in Andes, the

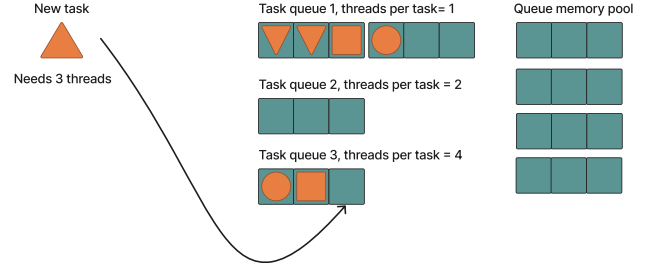


Fig. 3: Task enqueue inside of the current insertion multi-queue. Each shape denotes a different task type: the same task type can have a differing number of threads required, and multiple copies of the same task type can appear. Based on its size, the task is routed to the task queue with the *size* that can hold it. The memory available to the task queues comes from a shared memory pool. When a queue grows too large, another block of memory is taken from the pool and added to the end of the queue.

entire graph is fused into three kernels which run in order, with all tasks executing as soon as their dependencies are met.

Within a phase, tasks are executed in parallel on the GPU, while inter-phase boundaries provide global synchronization. This structure allows Andes to avoid fine-grained synchronization and instead leverage kernel boundaries for ordering and memory consistency. The executor is responsible for task construction, task execution, result accumulation, dependency management, and cleanup. These operations are organized around a set of task queues, which form the core abstraction for managing work.

A. Task queues

Task queues are the core abstraction used to organize and schedule work on the GPU. For realizing task queues, we implement buffered ring queues as these provide constant time read/write access to items in the queue. These queues solve two challenges: 1) in each round, we need to quickly determine the amount of parallelism required and 2) different tasks have different degrees of parallelism: one task may require only one thread, while another task may have a million components, each of which requires a block of threads.

To support both of these cases efficiently, we parameterize both tasks and task queues by **size**, corresponding to the number of threads required for execution. Sizes are rounded to the nearest power of two, and a queue with $size = 2^k$ is responsible for tasks requiring up to 2^k threads. This design enables tasks that require a large number of threads to be enqueued in constant time: enqueueing a task that requires a billion threads is no more difficult than enqueueing a task that requires one thread.

The task queues all share memory in the form of a global memory pool. When a queue grows too large, a new buffer of space is pulled from the ring queue and appended to the task queue. This allows individual task queues to scale efficiently

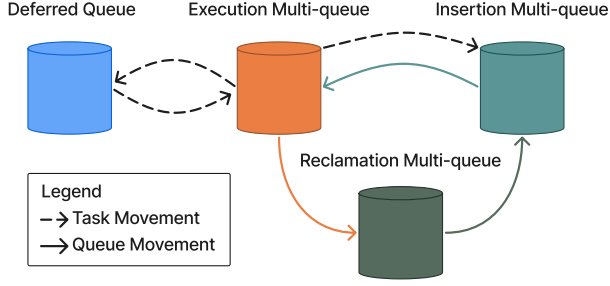


Fig. 4: Internal Structure of Andes. In each round, tasks from the execution queue are executed. Any new tasks that are created are either a) moved to the Insertion multi-queue to be processed in the next round or b) sent to the deferred queue to be launched when dependencies are met. At the end of each round of execution, the multi-queues shift to the next phase: the execution queue cleans up its resources, the insertion queue becomes the new execution queue, and the reclamation queue is ready to accumulate new tasks.

while also limiting the total memory use of the executor. The process of insertion and resize can be seen in Figure 3: A task that requires 3 threads is enqueued into the queue with task size = $2^2 = 4$. In this example, queue 1 required additional space and has been allocated an additional buffer from the memory pool, allowing it to hold 4 items.

During a given round of execution, a queue will hold one of three roles:

- **Execution:** Processing existing tasks.
- **Insertion:** Accumulating newly created tasks.
- **Reclamation:** Freeing resources to the system. The queue releases all task allocations and releases internal blocks back to the memory pool.

A queue will hold one role for an entire phase of execution before moving to the next phase in the cycle (see Figure 4).

Multi queue. To eliminate contention and enable efficient phase transitions, Andes organizes task queues into groups called *multi queues* which span all supported task sizes, i.e. a multi queue with 32 queues can support tasks that need anywhere from 2^0 - 2^{31} threads.

The executor maintains three types of multi queues, which rotate roles across phases:

- an *execution queue*, for reading/executing active tasks,
- an *insertion queue*, for storing newly generated tasks,
- a *reclamation queue*, for storing/reclaim resources from completed tasks.

During task execution, every task queue in the execution multi-queue that has tasks will issue one kernel launch using CUDA dynamic parallelism. This launch will process all tasks stored in the queue simultaneously.

The rotation of queues can be seen in Figure 4. Each multi queue holds its role for one phase of execution and then rotates to serve the next role in the cycle. The rotation ensures that

queue operations are phase-separated and only need a single atomic operation to claim a queue position - making accesses entirely contention-free.

Due to the bulk-synchronous design, all read-write transitions occur between kernel launches. This allows Andes to rely on implicit global synchronization and memory visibility guarantees provided by kernel completion (augmented with `__threadfence()`), eliminating the need for fine-grained synchronization within kernels.

Deferred execution and deferred queues. Andes distinguishes between *executable* tasks, whose dependencies are satisfied, and *deferred* tasks, which depend on unresolved dependencies. Dependency tracking is done through the use of **futures** which are handles to the results of other tasks. Futures include both a position to store the result and a flag to signal when the result has been written. If a task depends on at least one unresolved future, it is marked as deferred. Executable tasks are immediately enqueued into the multi queues for execution. Deferred tasks are placed into a separate structure called the *deferred queue*. This design enables Andes to defer dependency resolution without requiring fine-grained synchronization or atomic counters.

The deferred queue consists of two internal queues and is designed to defer dependency resolution until the executor reaches a point when all currently executable work has been completed. At this point, deferred tasks are revisited, and those whose dependencies are satisfied transition into executable tasks.

This design avoids the need for fine-grained synchronization or atomic dependency tracking during task execution. Instead, dependency resolution is performed in batches between phases, enabling efficient support for constructs such as `reduce()` and `aggregate()` without requiring prior knowledge of task counts. Tasks that remain unresolved are re-enqueued into the deferred queue, and the internal deferred queues swap roles, allowing memory associated with completed tasks to be reclaimed.

Memory management. All task allocations and internal memory management are handled by the Gallatin [13] allocator. Gallatin is an ensemble allocator that routes allocations through a hierarchy of fixed-size slabs using vEB trees.

We use Gallatin for two reasons. First, it supports arbitrary-sized allocations, allowing Andes to manage both user data and internal structures without specialized handling. Second, it coalesces allocation requests at the warp level, i.e., when multiple threads request memory of the same size, allocations are served collectively using a single atomic operation. All task allocation and deallocation are handled automatically using Gallatin. This is why, in Figure 1, Andes has no measured cleanup cost: cleanup occurs simultaneously with task execution.

Support for high-performance bulk allocation is particularly beneficial for task-parallel workloads, where bursts of task creation occur (e.g., after a `for_each()`). In such cases, Gallatin *coalesces* allocation requests, significantly reducing allo-

cation overhead compared to allocators such as Ouroboros [20] which require at least one atomic per allocation.

B. Eliminating task virtualization overheads

While Andes targets irregular workloads, many applications reduce to fully data-parallel execution. For example, triangle counting on an input graph in compressed sparse row (CSR) format can be implemented by assigning one thread per edge, requiring no dynamic tasking or dependency management.

In such cases, representing work through the Andes tasking abstraction can potentially introduce unnecessary overheads due to dynamic dispatch and task scheduling.

To avoid these unnecessary overheads, Andes automatically detects when execution reduces to a single, large, dependency-free task. When this condition is met, the executor bypasses the tasking infrastructure and directly launches a specialized kernel corresponding to the task.

This optimization effectively de-virtualizes execution. The task is converted back into an inlinable function within a monolithic kernel, restoring the compiler’s ability to perform aggressive optimizations such as inlining and specialization. This optimization allows Andes to match the performance of hand-written CUDA kernels on regular workloads, while retaining the flexibility to efficiently execute dynamic task graphs when needed.

IV. PROGRAMMING MODEL

We now describe Andes task API that application developers employ to produce task DAGs that run efficiently on GPUs while avoiding GPU-specific constraints that make programming difficult. We introduce the **flow** API in Andes to allow developers to easily translate complex parallel task DAGs to simple linear functions.

A. Andes API

The main operator in this API is **flow()**, which initiates a chain of linear work. Every step in this chain is a *functor*, an object which represents a function call and its arguments packed together. Functors can be natively run in the Andes executor, but must be templated to accept a context `Ctx` type as the first argument. The context enables interaction between tasks and the executor and allows functions to dynamically allocate shared memory, determine task ID and block size, and submit child tasks to the executor via `subtask()`.

A flow extended via a set of pipelined operations. The operations that can be pipelined in a flow are detailed below:

- **transform(Functor stateless_func)**: applies a stateless functor to the current state, forwarding the result to the next stage. The functor must be a pure function of its inputs, enabling fusion with neighboring tasks to amortize scheduling overhead.
- **for_each(Functor calculate_dimensions)**: `for_each` is the main tool for implementing parallelism in Andes. This flow task applies the function `calculate_dimensions` to the current state, returning a tuple specifying the fan-out for the next stage in the

flow. The state itself is unchanged. All downstream tasks are then spawned in the next iteration in parallel, e.g., a returned fan-out of 10 results in ten independent executions of subsequent stages of the flow. The returned value can be a multidimensional vector of any size.

- **Reduce(Functor reduction_functor)**: reduces inputs to a single value. This can be any user defined functor and take in any number of arguments. For convenience, we define a helper that implements CUDA atomic operations for numeric types.
- **Recurse(Functor transformationFunctor)**: semantically identical to `transform`, but disables fusion, allowing the task body to safely invoke `context.subtask()` for internal recursion. Once the recursive chain terminates, execution resumes at the next stage of the flow.
- **Loop(Flow body, Functor stoppingCondition)**: repeatedly executes a subflow until the stopping condition evaluates to false, providing structured iteration.
- **block<template blockSize>()**: signals to the executor that a specific block size should be used. The executor will launch one block of the specified size for each task: if `for_each` specifies 10 tasks with a block size of 128, 1,280 threads will be issued to process that task.

Advanced operators. Andes flow API also includes `Filter`, `Aggregate`, `Aggregate_by_key`, `Subflow`, `Carry`, and `Readd` operations for more complex specification. We omit the details of these operations due to space constraints.

Pseudocode examples. Two examples of using Andes API can be seen in Figure 5 and Figure 6. In Figure 5, calculation of the N^{th} Fibonacci number is done in two phases. In the first phase, Andes uses the `recurse` flow module to handle the parallel fan-out: all tasks re-enqueue themselves as children until they reach $N=1$ or $N=0$. After the recursion reaches base case, every thread returns their current N value which are then reduced in parallel.

For space, we omit the device functor code from Figure 6 to instead focus on the host API. This workload features a more advanced flow usage via a multi-step `for_each` to `reduce`. In the loop body, one block is spawned for every vertex. The threads in this block calculate the new weight for the vertex and do a local reduction to determine the max error. These max errors are then reduced across all threads to determine the maximum error of the whole system, and the work buffers are swapped. A `while_loop` is used to check convergence and re-issue work. While the error remains above the threshold, the loop body is re-issued. This allows for the work to be performed iteratively without host intervention, minimizing turnaround times.

Futures. The result of a flow is always a future that will hold the result tuple from one thread when the flow is finished. These futures track flow completion and mark if the flow produced output. Futures can be fed as the input to another flow to introduce a dependency; when that flow is put into the


```

1 struct FibTask {
2     template <typename Ctx>
3     __device__ CUDA_Tuple<int> operator()(Ctx& ctx, int n) {
4         if (n < 2) return wrap(n); //base case
5         ctx.subtask(FibTask{}, n - 1); //launch children
6         ctx.subtask(FibTask{}, n - 2);
7         ctx.kill_task(); //terminate task
8         return wrap(0);
9     }
10 };
11 //accumulate all results atomically, sum starts at 0
12 struct SumReducer {
13     __device__ int accumulate(int* acc, int value) {
14         return atomicAdd(acc, value);
15     }
16     __device__ static int get_default() { return 0; }
17 };
18 int main {
19     auto executor = andes::init_executor();
20     //create flow and mark it as returning a result
21     auto f = flow(executor)
22         .recurse(FibTask{})
23         .reduce(ReducerWrapper<SumReducer>{})
24         .finalize_with_result();
25     //calculate 30th fibonacci
26     // handle populated after execution
27     auto result_handle = f.submit(30);
28     executor->launch_tasks();
29 }

```

Fig. 5: Source code for calculating the N^{th} Fibonacci number with Andes. The recursive structure of the task is encapsulated in `FibTask` which enqueues itself. Once $N < 2$, tasks move to the reducer step of the flow which accumulates results.

executor, it will wait in the deferred queue until the future is written to, after which it will be automatically enqueued into the main execution pipeline.

B. Optimizing flows

The flow API is designed to be both simple and flexible for users but also expose necessary type information for aggressive optimization. We now discuss the optimizations that are automatically applied during flow construction.

Flow fusion. Every task launch comes with non-negligible overhead. When possible, we want to **fuse** tasks into one launch of the executor. This reduces the overhead of the scheduler, allows for more work to be done in one iteration, and enables users to break programs into smaller logical components without affecting performance.

By construction, tasks are either fusable or not fusable. When the flow is finalized, we condense the tasks into *fused groups* using the following rule: each fused group starts with zero or more fusable tasks and ends with an unfusable task. The final fused group in the flow can end with a fusable task if there are no unfusable tasks at the end.

The following tasks are fusable: transforms, filters, and metadata setup/teardown (discussed in the next section).

C. Setup/Teardown tasks

Some flow operations (e.g., `reduce`) require auxiliary work such as allocation/initialization before parallel execution and finalization afterwards. Andes automatically inserts these setup and teardown tasks during flow construction.

When injecting, the metadata tasks are fused with adjacent operations when possible to minimize overhead. This allows Andes to support advanced features like dynamic vector aggregation while keeping the internals largely invisible to the programmer.

D. Non-divergent task optimization

Operations that collapse parallelism (`reduce`, `aggregate`) require coordinating task completion across threads. By deferring the completion of these operations, Andes guarantees that all contributing threads will reach the collapsing operation before the final value is read out. In the general case, Andes uses a single atomic to ensure that only one thread performs finalization.

When parallelism is non-divergent (i.e., no threads are filtered and all threads reach the operation together), Andes eliminates this overhead by deterministically assigning a single thread to perform completion. This avoids the overhead of atomics entirely.

E. Data dependent parallelism

In some programs, the exact task graph structure cannot be known a priori and must be discovered during execution. An example of this is the **Fibonacci** series: different child calls will terminate at different points in the execution, and the task DAG is heavily imbalanced. Baking these recursive operations into a DAG incurs a high performance cost, as the parallelism must be traced out on **host**; this limits the scalability as the graph becomes larger.

Since the Andes executor uses virtualization to handle task specialization, there are no limitations to what kind of task can be enqueued next. This is expressed to the user via the **recurse** module, which allows users to dynamically construct and enqueue new tasks **during** flow execution. In the Fibonacci Andes flow (see Figure 5), the problem subdivision can be expressed recursively: if the problem size N is larger than 2, the system enqueues both children recursively; otherwise, the task moves to the next step to be reduced. This allows for the parallelism to be explored efficiently without requiring the construction of the full task DAG.

V. EVALUATION

In this section, we evaluate the performance of Andes across several scientific computing applications. For each application, we compare against CUDASTF, a state-of-the-art GPU runtime system. Additionally, we compare against the state-of-the-art custom GPU implementation, or mark if a suitable implementation was not available. We perform experiments

```

1 //Pagerank loop body
2 auto body = flow(executor)
3   .for_each(SpawnVertices{}) //for each vertex
4   //spawn block of B threads, reduce on max error
5   .transform(GatherRanks{}).block<B>()
6   .reduce(ReducerWrapper<MaxReducer>{}).block<B>()
7   .transform(SwapBuffers{}); //swap buffers

8 //main flow - loop until error < threshold
9 auto f = flow(executor)
10  .while_loop(NotConverged{1e-6f}, body)
11  .finalize();

12 f.submit(n_verts, 1.0f);

```

Fig. 6: Host code for building the Pagerank flow. The flow is composed of two components: a subflow that acts as a loop body, and a main flow that iteratively calls the subflow until convergence is reached.

across two Nvidia GPUs, H200 with 143 GB HBM3e and 16,896 CUDA cores and Blackwell RTX 6000 Pro with 96 GB GDDR7 and 24,064 CUDA cores.

A. Evaluation benchmark

We evaluate Andes on the following applications along with their custom baseline implementations:

- ***N*-Queens**: count how many different ways N queens could be placed on an $N \times N$ chessboard such that no queen can attack any other queen. The baseline implementation from Yao et Li [21] is a hand-built solver that uses an unrolled hand-written PTX loop and a custom stack architecture to process N -Queens states. We test on N queens = 15.
- **SpGEMM**: sparse matrix multiply of a matrix by its transpose. The baseline implementation TileSpGEMM [14] is a custom SpGEMM kernel that breaks down each matrix into 16×16 tiles and processes them using a three-phase symbolic-symbolic-numeric approach.
- **Graph Algorithms**: We employ four graph algorithms in our benchmark, minimum spanning tree (MST), pagerank (PR), triangle counting (TC), and single-source shortest path (SSSP). We compare against Gunrock [18], a state of the art for GPU graph algorithms and contains optimized implementations of many graph algorithms inside of a custom wavefront execution system built on Thrust.
- **Prime Sieve**: Use the sieve of Eratosthenes to count the number of primes in a range. For the Baseline, we compare to CUDASieve [17], an open-source sieve of Erathosenes written in CUDA. We set N to be 10^{12} .
- **Karatsuba**: perform $O(n^{1.585})$ multiplication of two very large numbers by recursively subdividing the problem. We did not find a working GPU implementation. Therefore, we compare to ParlayLib [5] running with 128 CPU threads. We set the number of bits N to 2^{25} bits.
- **Radix Tree Construction**: bulk radix-tree construction with 100 million random keys each between 2-15 characters long.

We did not find a working GPU implementation. Therefore, we compare to ParlayLib [5] running with 128 CPU threads.

- **Smith Waterman**: $O(n^2)$ string matching algorithm. An anti-diagonal wavefront is used to parallelize tasks. We employ the optimized implementation from Juggler [4] as the baseline, and match the string size from the Juggler paper at 46,080 bases in each string.
- **MiniWeather** MiniWeather [15] is a toy simulation of a weather model used to show parallelization speedups of dense 2D Euler equations. It implements a two-direction Runge-Kutta time stepping over a finite 2D grid. In Augonnet et al. [1], the authors implement miniWeather with CUDASTF, but this implementation is closed source and not publicly available. We re-implement this benchmark for CUDASTF and include a variant that uses raw CUDA kernels to process the grid as well.

Dataset. The graph benchmarks were run on the Orkut social graph from SNAP [12]. This is a sparse graph with 3,072,441 vertices and 117,185,083 edges. For problems which require edge weights, we augment the graph with a set of random edge weights between 0-100. For SpGEMM, we use the consph from SuiteSparse [8]. This is a symmetric matrix of 83,334 rows and columns and 6,010,480 nonzeros originally described in Williams et al. [19]. For Smith waterman, the dataset is randomly generated using the generator from Belviranli et al. [4].

CUDASTF. Along with the non-tasking state-of-the-art, we also compare against CUDASTF. CUDASTF is a tasking system built on top of CUDA graphs and CUDA Dynamic Parallelism 2. In CUDASTF, tasks are baked into a runtime graph execution on host for efficient GPU execution. Unlike Andes, CUDASTF can scale to multiple nodes using the NVIDIA Collective Communications Library (NCCL).

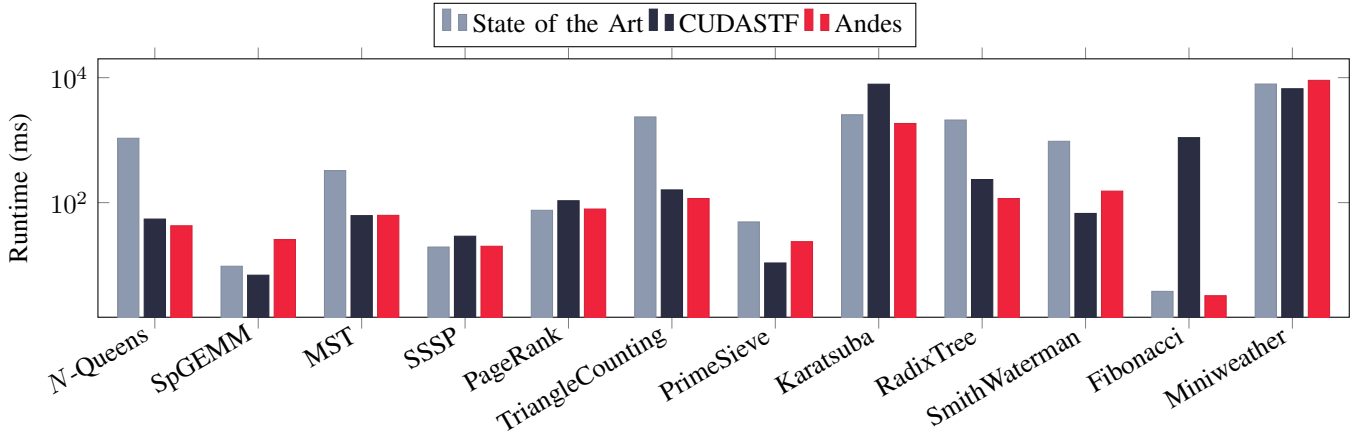
B. Application results

Figure 7a and Figure 7b show the running time of Andes, CUDASTF, and the baseline implementations across several applications.

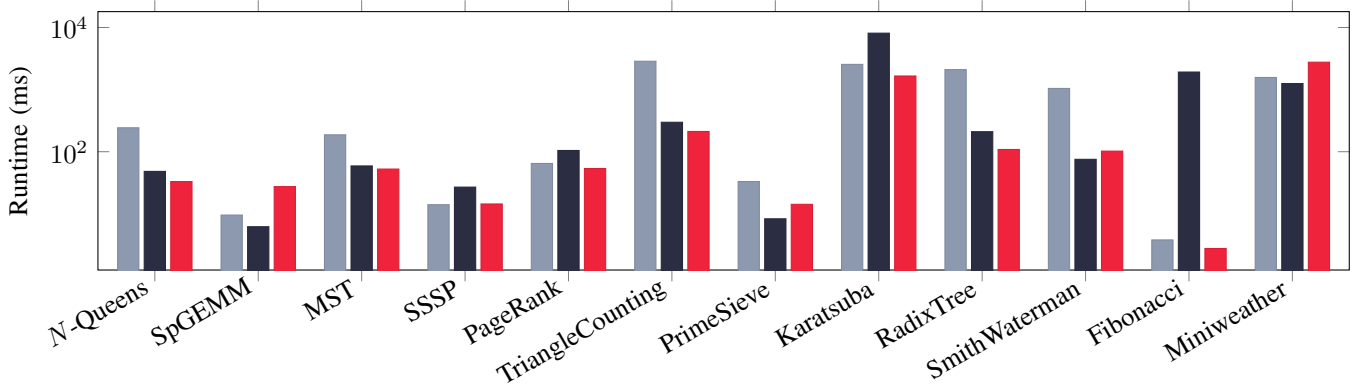
***N*-Queens.** For Andes, we implement a recursive N -queens solver with two optimizations from the custom solver: using a bitboard to store the state and mirroring the first row with a multiplier.

CUDASTF cannot efficiently process the benchmark as-is due to GPU task recursion being constructed on host - attempting to build out the full graph for N -Queens=15 took over an hour and timed out in our experiments. To implement N -Queens with CUDASTF, we implement N -Queens with a pair of statically allocated buffers for wavefronts. This extra scaffolding allows CUDASTF to process N -Queens but is very brittle. We set the buffer size to 50 million bitboards to ensure it does not run out of memory.

On the N -Queens benchmark, Andes’s bitboard solver is $25\times$ faster than the custom solver. This is because the custom solver uses a manual stack and hand written PTX code which cannot be optimized by the compiler. In comparison,



(a) Nvidia H200 results.



(b) Nvidia Blackwell RTX 6000 Pro results.

Fig. 7: Performance results of Andes, CUDASTF, and the state-of-the-art custom solution for twelve HPC applications on two Nvidia GPUs (H200 and Blackwell RTX 6000 Pro). Times given are running times in milliseconds, lower is better.

Andes benefits from automatic load balancing and device-side scheduling. CUDASTF implements the same algorithm, but must bounce to host in every iteration to launch the next round. This host-device communication results in a 28% slowdown compared to Andes.

SpGEMM. On SpGEMM, Andes is unable to match the performance achieved by TileSpGEMM [14], the current state of the art for SpGEMM. This is because the SpGEMM kernel runs in three phases of symbolic, symbolic, numeric. In this structure, the amount of parallelism available at each phase of the SpGEMM kernel can be determined a priori. This means that the scheduler and task system create pure overhead, as the kernels can be launched efficiently without Andes. On the Consph matrix, this overhead translates into a 62% slowdown compared to TileSpGEMM.

On the other hand, this kind of predefined structure is the ideal use case for CUDASTF- tasks have simple dependencies, known sizes, and a known order. CUDASTF is able to compress all three phases into an efficient CUDA graph for execution and improves over the custom solver, performing

the SpGEMM multiply in only 6.9 ms, a 38% speedup over the custom solver.

MST. For minimum spanning tree (MST), we implement parallel Boruvka’s algorithm [7] to calculate the MST efficiently. For MST implementation, we rely on Andes’s implicit memory synchronization: a lazy write of the update flag tracks whether or not all nodes have converged. We parallelize across the edges, and assign each thread to handle 128 edges. This layout handles both small and large degree vertices efficiently: small vertices are owned entirely by one thread, allowing for a fast update, while large vertices are parallelized across multiple threads, preventing tail latency due to one thread processing their high degree.

These optimization result in a kernel that is over $4.06\times$ faster than Gunrock [18] on the Orkut graph. CUDASTF can also implement this edge-parallel algorithm. On the H200, their performance is almost identical, while on the RTX 6000 Andes is 12% faster.

SSSP. For single-source shortest path (SSSP), we implement a recursive algorithm where vertices are claimed with an

AtomicMin instruction. This algorithm is slower than Gunrock due to the need to atomically determine which thread has claimed a node: Gunrock allows all threads to claim and then de-duplicates using Thrust sort. As a result, Andes SSSP runs slower, completing the Orkut graph in 20.023 milliseconds whereas Gunrock can complete it in 10.46 milliseconds. CUDASTF’s performance is 29 milliseconds on this benchmark as it suffers the downsides of both atomic claims and host-side launches.

PageRank. For the pagerank (PR), we implement a blocked algorithm with two buffers. In each iteration, a block of 16 threads is assigned a subset of vertices in the graph. For each vertex in their subset, the block cooperatively strides across the edges of the vertex to calculate the new weight, then perform a local reduction and have the first thread perform the update. Using the Andes, it is trivial to optimize for the ideal block size and stride necessary to fully saturate the GPU compute. Iterating through the block space gives us a final performance of 76.136 milliseconds compared to Gunrock at 75.3 milliseconds.

Triangle Counting. For triangle counting (TC), we read in the graph in CSR format and perform edge intersection starting from the two points defined by every existing edge. This variant of TC is trivial to parallelize - it runs in one iteration, and the number of threads needed is the number of edges in the original graph. We implement the intersection variant of this algorithm and assign a single thread to calculate each intersection list. With Andes handling the scheduling of each task and performing the reduction in shared memory, we can count triangles in 116.378 milliseconds - over $20\times$ faster than Gunrock on the Orkut social graph. CUDASTF is also able to efficiently parallelize this algorithm and achieve a performance of 159.8 milliseconds on the same graph.

Prime Sieving. For prime sieving, we implement the traditional Sieve of Eratosthenes [10]. The set of primes up to $\sqrt{N}$ are precalculated, and then used to sieve individual segments of the bitmap, with each segment being assigned to one thread block. For this problem, the number of tasks needed and their size can be known a priori. This gives an advantage to CUDASTF as it can bake the problem into a single CUDA kernel. CUDASTF is able to complete the size of 100 million results in only 10.82 milliseconds, whereas Andes takes 23.7 milliseconds. CUDASieve, the state of the art, has additional logic used to scale to arbitrarily high primes. This overhead gives them a runtime of 49.1 milliseconds.

Karatsuba. For Karatsuba, we implement the original Karatsuba algorithm using the `recurse()` to spawn both subtasks. The authors were unable to locate another GPU implementation of the Karatsuba algorithm, so we compare to the parlayLib [5] implementation being run with 128 threads. Compared to the CPU implementation, the Andes algorithm can multiply two 32 megabit numbers over $1.5\times$ faster than the CPU implementation.

Algorithms like Karatsuba are the most difficult for CUDASTF to process, as the recursive graph depth varies based on

the problem and subproblem size. This means that structure of the graph cannot be efficiently flattened like in N -Queens. To process Karatsuba, CUDASTF must run through the upper level parallel structure of the problem on host in order to construct the task graph. This approach limits the scalability of their approach: as the number of bits in the problem 2^{25} , CUDASTF actually becomes slower than parlayLib, with a runtime of 7.8 seconds.

Radix Tree. For the radix tree, we implement a bulk synchronous construction algorithm. We use a blocked all nearest small values (ANSV) algorithm and parallel prefix sums to efficiently identify where in the radix tree a string should be placed. We compare against the implementation built on CPU by ParlayANN when used to construct 100 million random strings with length between 2-15. For construction of this dataset, parlayANN takes 2.1 seconds with 64 threads. Andes can perform the same construction in only 116 ms, whereas CUDASTF can perform the same construction in 234 milliseconds. The difference in launch cost is due to two factors: Andes can schedule kernels on device without host intervention and 2) Andes’s flow is able to fuse most steps in radix tree construction, cutting down on the number of launches needed.

Fibonacci. For Fibonacci, we implement the algorithm recursively, following the code in Figure 5. Calculating the 26th Fibonacci number recursively requires 1,664,079 recursive calls. Thanks to the `recurse` module, Andes is able to support all of these calls in just 26 kernel launches. CUDASTF, on the other hand, must unroll the entire structure on host. This results in an over $1500\times$ performance difference on this benchmark, highlighting the data-dependency limitation Andes is built to overcome. ParlayLib [5] can also efficiently process Fibonacci, and is only .6 milliseconds slower than Andes despite being run on CPU.

MiniWeather. MiniWeather is another application that is most efficiently expressed as a CUDA graph: the calculations are dense, structured, and run in a predictable order. For this benchmark, using Andes introduces some overheads: On the H200, the raw CUDA kernels solve a 2000×2000 grid in 7.9 seconds, and CUDASTF improves on the baseline, solving in 6.646 seconds. Due to the overheads of the executor, Andes is actually slower on this benchmark, taking 9.047 seconds to perform the same accumulation.

C. Microbenchmarks

We now evaluate the performance of various design optimizations such as task fusion, dynamic dispatch, and block tuning in Andes.

Task fusion. In this benchmark, we test the performance of a simple operator as it is repeatedly applied to a dataset. For this benchmark, we chose to apply the Collatz conjecture step ($x/2$ if even, $3x - 1$ if odd) as it can be performed very quickly, allowing us to highlight the overhead of the tasking system. For this test, we use an array containing 100 million sequential numbers, starting at 1.

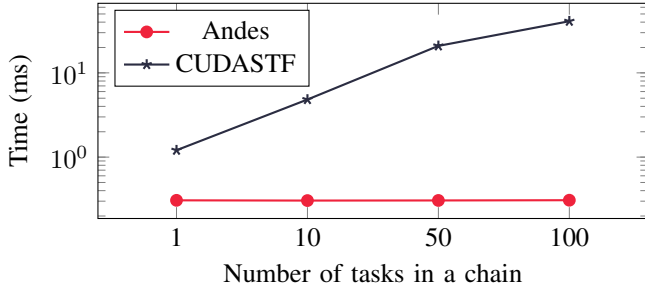


Fig. 8: Execution time comparison between Andes and CUDASTF as the number of tasks grow. Andes can fuse these tasks into one kernel operation, whereas CUDASTF must construct a graph node for each operation.

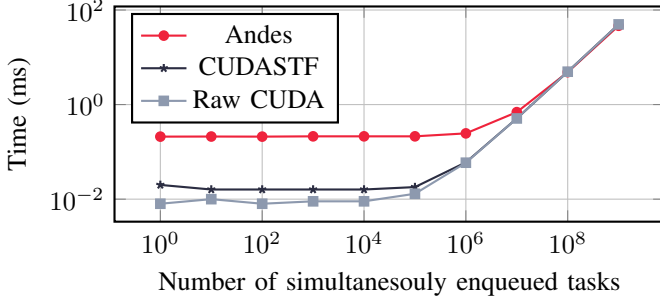


Fig. 9: Runtime of a single kernel as a measure of the number of tasks enqueued simultaneously.

Figure 8 shows the performance change as we increase the number of tasks chained together, starting at one task and increasing to 100 tasks serially chained in the DAG.

As the number of steps grows, the complexity and runtime of the CUDASTF CUDA graph grow as well: applying the task one time to every number takes only 1.2 milliseconds. However, as the number of tasks grows, the cost of applying the task in CUDASTF scales linearly, up to a 40 millisecond runtime for applying the task 100 times to every numbers.

Andes, on the other hand, is able to detect and fuse the tasks together. This results in all transforms being injected as linear steps in one kernel. As each step is relatively cheap, the costs of kernel launches dominate: increasing the number of steps does not increase the measured runtime as only one kernel launch occurs.

Task launch scalability. Figure 9 shows the time taken to execute a task graph as a function of the number of simultaneously enqueued tasks by the executor. As expected, issuing a CUDA kernel directly has the lowest latency, with a launch time of 8 microseconds for a single task. CUDASTF is a close second, with a launch cost of only 20 microseconds. The launch cost for Andes is significantly higher, at 200 microseconds for the kernel launch. The high launch latency of Andes is due to the generic structure of the Andes executor. Andes uses the deferred queue and virtual functions to properly process multiple flows, flows that depend on futures, and flows that are deferring operations or memory frees. These components

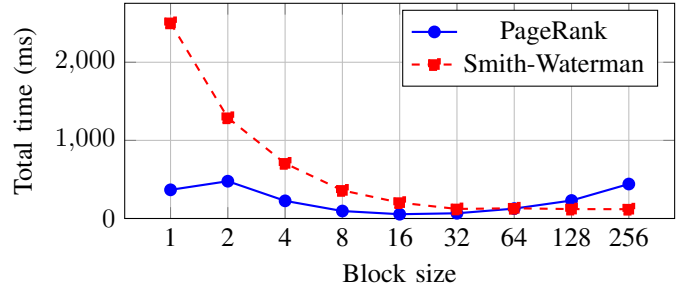


Fig. 10: Total runtime vs. block size for PageRank and Smith-Waterman in Andes.

provide greater flexibility and help Andes perform algorithms like MST and Karatsuba efficiently, but they have non-trivial overheads that result in the high start latency observed here.

All three systems scale roughly the same as the number of tasks increases. As the throughput of processing tasks begins to dominate, task launch overheads are amortized in Andes. All systems offer roughly identical performance above 10 million tasks.

Block size. The result of varying the block size per-task is presented in Figure 10. Varying the block size leads to a significant change in performance - PAGERank achieves the lowest runtime when the block size is 16, and the best block size is $8.7\times$ faster than the worst block size. For Smith-waterman, the best tile size is 32, and there is a $21\times$ performance difference between the best and worst configuration.

VI. CONCLUSION

We present Andes, a general-purpose and high-performance runtime for dynamic parallelism on GPUs. Using dynamic dispatch and task fusion, Andes can efficiently group and launch tasks and construct the task DAG on the fly, eliminating the need to produce the task structure on host.

In real-world benchmarks, Andes exceeds the performance of the non-tasking state of the art in 9 out of 12 benchmarks, and exceeds the performance of CUDASTF in 8 out of 12 benchmarks. On benchmarks that need dynamic parallelism, Andes is the fastest tasking system, even when CUDASTF is augmented with a task buffer like in *N*-Queens. On benchmarks where the work bound is static, CUDASTF is the fastest as all operations can be baked into a CUDA graph with known topology.

Andes introduces a truly dynamic tasking framework. We show that dynamic tasking is a crucial feature for solving many kinds of problems efficiently, though it introduces slight overhead compared to CUDASTF for some static problem types. Andes enables developers to easily write high performance GPU applications, allowing them to tackle dynamic problems that were previously untenable on GPUs.

ACKNOWLEDGMENT

We employed ChatGPT [16] to identify grammatical errors and typos and also get suggestions for improving the clarity in certain parts of the paper.

REFERENCES

- [1] C. AUGONNET, A. ALEXANDRESCU, A. SIDELNIK, AND M. GARLAND, *CUDASTF: Bridging the gap between CUDA and task parallelism*, in Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC), IEEE, 2024, <https://doi.org/10.1109/SC41406.2024.00049>.
- [2] C. AUGONNET, S. THIBAUT, R. NAMYST, AND P. WACRENIER, *Starp: a unified platform for task scheduling on heterogeneous multicore architectures*, Concurrency and Computation: Practice and Experience, 23 (2010), p. 187–198, <https://doi.org/10.1002/cpe.1631>, <http://dx.doi.org/10.1002/cpe.1631>.
- [3] J. BACHAN, D. BONACHEA, P. H. HARGROVE, S. HOFMEYR, M. JACQUELIN, A. KAMIL, B. VAN STRAALEN, AND S. B. BADEN, *The upc++ pgas library for exascale computing*, in Proceedings of the Second Annual PGAS Applications Workshop, PAW17, New York, NY, USA, 2017, Association for Computing Machinery, <https://doi.org/10.1145/3144779.3169108>, <https://doi.org/10.1145/3144779.3169108>.
- [4] M. E. BELVIRANLI, S. LEE, J. S. VETTER, AND L. N. BHUYAN, *Jugler: A dependence-aware task-based execution framework for GPUs*, in Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP), Vienna, Austria, 2018, ACM, pp. 54–67, <https://doi.org/10.1145/3178487.3178492>.
- [5] G. E. BLELLOCH, D. ANDERSON, AND L. DHULIPALA, *Parlaylib - a toolkit for parallel algorithms on shared-memory multicore machines*, in Proceedings of the 32nd ACM Symposium on Parallelism in Algorithms and Architectures, SPAA '20, ACM, July 2020, p. 507–509, <https://doi.org/10.1145/3350755.3400254>, <http://dx.doi.org/10.1145/3350755.3400254>.
- [6] R. D. BLUMOF, C. F. JOERG, B. C. KUSZMAUL, C. E. LEISERSON, K. H. RANDALL, AND Y. ZHOU, *Cilk: an efficient multithreaded runtime system*, ACM SIGPLAN Notices, 30 (1995), p. 207–216, <https://doi.org/10.1145/209937.209958>, <http://dx.doi.org/10.1145/209937.209958>.
- [7] O. BORŮVKA, *Über ein Minimalproblem*. Práce moravské přírodovědecké společnosti 3 (1926), 37–58 (1926), 1926.
- [8] T. A. DAVIS AND Y. HU, *The university of florida sparse matrix collection*, ACM Transactions on Mathematical Software, 38 (2011), p. 1–25, <https://doi.org/10.1145/2049662.2049663>, <http://dx.doi.org/10.1145/2049662.2049663>.
- [9] F. G. GUSTAVSON, *Two fast algorithms for sparse matrices: Multiplication and permuted transposition*, ACM Transactions on Mathematical Software, 4 (1978), p. 250–269, <https://doi.org/10.1145/355791.355796>, <http://dx.doi.org/10.1145/355791.355796>.
- [10] S. HORSLEY, *Κοῦκινος ἐπὶ τοῦ ὁμοῦ τοῦ Σ . or, the sieve of erathostenes. being an account of his method of finding all the prime numbers*, by the rev. samuel horsley, f. r. s., Philosophical Transactions (1683–1775), 62 (1772), pp. 327–347, <http://www.jstor.org/stable/106053> (accessed 2026-04-07).
- [11] T.-W. HUANG, D.-L. LIN, C.-X. LIN, AND Y. LIN, *Taskflow: A lightweight parallel and heterogeneous task graph computing system*, IEEE Transactions on Parallel and Distributed Systems, 33 (2022), p. 1303–1320, <https://doi.org/10.1109/tpds.2021.3104255>, <http://dx.doi.org/10.1109/TPDS.2021.3104255>.
- [12] J. LESKOVEC AND A. KREVL, *SNAP Datasets: Stanford large network dataset collection*. <http://snap.stanford.edu/data>, June 2014.
- [13] H. MCCOY AND P. PANDEY, *Gallatin: A general-purpose gpu memory manager*, in Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP '24, New York, NY, USA, 2024, Association for Computing Machinery, p. 364–376, <https://doi.org/10.1145/3627535.3638499>, <https://doi.org/10.1145/3627535.3638499>.
- [14] Y. NIU, Z. LU, H. JI, S. SONG, Z. JIN, AND W. LIU, *Tilespgemm: a tiled algorithm for parallel sparse general matrix-matrix multiplication on gpus*, in Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '22, ACM, Mar. 2022, p. 90–106, <https://doi.org/10.1145/3503221.3508431>, <http://dx.doi.org/10.1145/3503221.3508431>.
- [15] M. NORMAN, *miniweather*, 2020, <https://doi.org/10.11578/DC.20201001.88>, <https://www.osti.gov/doccode/biblio/45848>.
- [16] OPENAI, *Chatgpt: Conversational ai*, 2023, <https://www.openai.com/chatgpt>. Accessed: 2026-04-01.
- [17] C. SEIZERT, R. VAN BERGEN, K. WALISCH, AND S. TROISI, *Cud-asieve*. <https://github.com/curtisizeert/CUDASieve>, 2021.
- [18] Y. WANG, A. DAVIDSON, Y. PAN, Y. WU, A. RIFFEL, AND J. D. OWENS, *Gunrock: a high-performance graph processing library on the gpu*, ACM SIGPLAN Notices, 51 (2016), p. 1–12, <https://doi.org/10.1145/3016078.2851145>, <http://dx.doi.org/10.1145/3016078.2851145>.
- [19] S. WILLIAMS, L. OLIKER, R. VUDUC, J. SHALF, K. YELICK, AND J. DEMMEL, *Optimization of sparse matrix–vector multiplication on emerging multicore platforms*, Parallel Computing, 35 (2009), p. 178–194, <https://doi.org/10.1016/j.parco.2008.12.006>, <http://dx.doi.org/10.1016/j.parco.2008.12.006>.
- [20] M. WINTER, D. MLAKAR, M. PARGER, AND M. STEINBERGER, *Ouroboros: virtualized queues for dynamic memory management on gpus*, in Proceedings of the 34th ACM International Conference on Supercomputing, ICS '20, ACM, June 2020, p. 1–12, <https://doi.org/10.1145/3392717.3392742>, <http://dx.doi.org/10.1145/3392717.3392742>.
- [21] G. YAO AND Y. LI, *High-performance n-queens solver on gpu: Iterative dfs with zero bank conflicts*, 2025, <https://doi.org/10.48550/ARXIV.2511.12009>, <https://arxiv.org/abs/2511.12009>.